图像变换实验报告

宁尚哲 PB24000099

一、实验目的

本实验旨在实现基于控制点的图像变换算法，包括反距离加权（IDW）插值和径向基函数（RBF）插值两种方法。通过面向对象的设计思想，将变换算法封装为独立的类，并与现有的图像处理框架进行集成，实现用户交互式的图像变换功能。

二、设计思路

2.1 整体架构设计

采用面向对象的设计模式，基于继承和多态特性构建变换算法框架：

- **抽象基类Warper**：定义统一的变换接口，封装数学映射逻辑，与具体图像处理解耦
- **具体实现类**：IDWWarper和RBFWarper分别实现不同的变换算法
- **集成层**：WarpingWidget负责用户交互和UI显示，调用Warper类完成变换

2.2 IDW插值算法设计

IDW算法基于距离反比权重计算，核心思想是：距离目标点越近的控制点，其影响越大。算法分为三个层次：

1. **权重计算**：使用距离函数σ和归一化权重ω计算每个控制点对目标点的影响
2. **局部变换矩阵**：为每个控制点计算2×2变换矩阵Ti，通过最小化能量函数求解线性方程组Ti·A = B
3. **全局变换**：将所有控制点的局部变换加权组合，得到最终的坐标映射

关键数学公式：

- 距离函数：

$$\sigma(\mu, i, x, y) = 1/||p - pi||^{\mu}$$

- 归一化权重：

$$\omega(\mu, i, x, y) = \sigma(\mu, i, x, y)/\Sigma\sigma(\mu, j, x, y)$$

- 变换函数：

$$f(p) = \Sigma\omega(\mu, i, x, y) \cdot [qi + Ti \cdot (p - pi)]$$

2.3 RBF插值算法设计

RBF算法将变换分解为仿射变换和非线性径向基函数两部分：

1. **仿射变换部分**：

$$Ap = A \cdot p + b$$

   处理全局线性变换

2. **径向基函数部分**：

$$Rp = \Sigma \alpha i \cdot g(||p - pi||)$$

   处理局部非线性变形

3. **参数求解**：根据控制点约束，使用最小二乘法求解A、b和α

径向基函数采用改进的Wendland函数：

$$g(d) = \sqrt{(d^2 + ri^2)}$$

其中ri为控制点到其他控制点的最小距离。

2.4 神经网络（NN）变换算法设计

神经网络算法采用深度学习方法学习控制点之间的非线性映射关系，通过训练一个多层感知机（MLP）来近似图像变换函数。算法设计包含以下核心思想：

1. **网络架构设计**：采用三层全连接网络结构
   - 输入层：2个神经元（原始坐标x, y）
   - 隐藏层1：10个神经元，使用ReLU激活函数
   - 隐藏层2：10个神经元，使用ReLU激活函数
   - 输出层：2个神经元（变换后坐标x', y'）
   - 损失函数：均方误差（MSE）
2. **训练数据生成策略**：
   - 基础数据：用户选择的控制点对（原始坐标 → 目标坐标）
   - 数据增强：在控制点之间进行线性插值生成额外训练样本
   - 插值策略：在每对控制点之间生成3个插值点（t=0.25, 0.5, 0.75）
   - 目的：增加训练数据量，提高网络泛化能力，避免过拟合
3. **训练过程优化**：
   - 优化器：Adam优化器，自适应学习率调整
   - 学习率：初始0.001，最小学习率1e-5
   - 批次大小：动态调整，最大32，不超过数据集大小
   - 训练轮数：默认1000轮，可根据需要调整

    - 早停机制：通过学习率衰减避免过拟合
4. **变换推理机制**：
    - 前向传播：将原始坐标输入网络，直接输出变换后坐标
    - 边界处理：检查输出坐标是否在图像范围内，超出范围则跳过
    - 实时性：网络推理速度快，适合实时交互
5. **网络持久化**：
    - 保存机制：训练完成后可保存网络权重到文件
    - 加载机制：可从文件加载预训练网络，避免重复训练
    - 灵活性：支持不同控制点配置的网络复用

算法优势：

- 能够学习复杂的非线性映射关系
- 训练后推理速度快，适合实时应用
- 可扩展性强，可通过调整网络结构适应不同复杂度的变换
- 支持网络保存和加载，提高使用效率

算法局限性：

- 需要足够的训练数据，控制点过少时效果不佳
- 训练过程需要时间，目前定的训练次数上限可能比较小，当映射点数稍微多一些后发现限定次数内loss没有降到0，导致图像部分地方处理不够好。
- 对控制点分布敏感，分布不均匀时可能产生局部变形

2.5 空洞填充（Hole Filling）算法设计

空洞填充算法用于修复图像变换后产生的黑色空洞区域，通过搜索邻近已知像素并插值填充。算法设计包含两个核心模块：

1. **空洞检测机制**：
    - 判定标准：RGB三个通道均为0的像素判定为空洞像素
    - 扫描策略：遍历整个图像，记录所有空洞像素位置
    - 数据结构：使用vector存储空洞坐标，便于后续批量处理
2. **邻近像素搜索策略**：
    - **暴力搜索方法**（基础实现，不好用）：
        - 搜索范围：以空洞像素为中心的矩形区域（半径可调）
        - 遍历方式：双重循环遍历搜索范围内的所有像素
        - 筛选条件：非空洞像素且在图像边界内
        - 时间复杂度：$O(n \cdot r^2)$，其中n为空洞数量，r为搜索半径
    - **Annoy加速方法**（优化实现）：
        - 空间索引：使用Annoy库构建2D空间索引结构
        - 索引构建：将所有已知像素坐标添加到Annoy索引中
        - 搜索策略：查询最近邻像素，限制在指定半径内
        - 性能提升：时间复杂度降至$O(n \cdot \log k)$，k为已知像素数量
        - 适用场景：大尺寸图像、大量空洞像素时性能优势明显
3. **颜色插值算法**：
    - **IDW插值方法**（method=0）：
        - 权重计算：

$$w_i = 1/d_i^p$$

，其中d_i为距离，p为幂次（默认2）

- 颜色计算：

$$C = \Sigma(w_i \cdot C_i)/\Sigma w_i$$

，C_i为邻近像素颜色

- 特点：平滑过渡，适合自然图像
- 参数调节：搜索半径控制影响范围，幂次控制权重衰减速度

- **最近邻方法**（method=1）：
  - 选择策略：选择距离最近的已知像素的颜色
  - 实现简单：直接复制最近像素的RGB值
  - 特点：保持原始纹理，适合边缘锐利的图像
  - 计算效率：无需权重计算，速度更快

4. **边界与异常处理**：
   - 距离阈值：搜索半径限制在0-10像素，避免过度填充
   - 权重归一化：防止除零错误，距离过近时设置极大权重
   - 颜色截断：插值结果限制在[0,255]范围内
   - 空洞保护：无邻近像素时保持黑色，避免错误填充
5. **算法优化策略**：
   - 批量处理：先收集所有空洞和已知像素，避免重复遍历
   - 索引复用：Annoy索引全局构建一次，所有空洞查询共用
   - 内存优化：使用引用传递避免图像数据拷贝
   - 并行化潜力：不同空洞的填充可并行处理

参数调优经验：

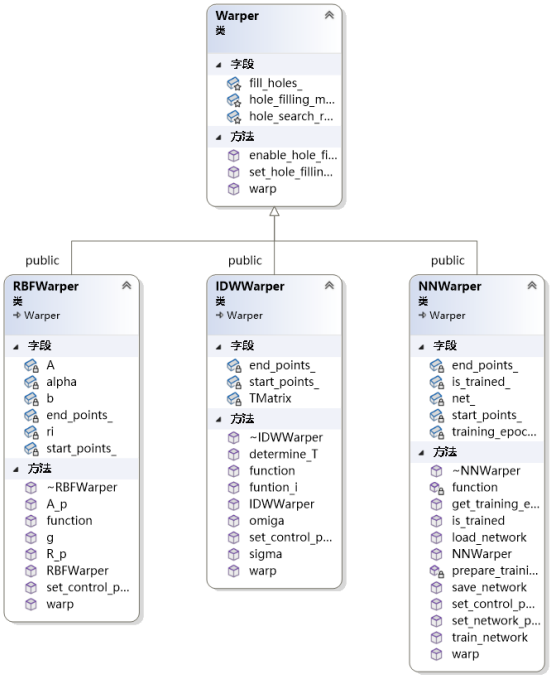
- 搜索半径：1-2像素适合细节保持，3-5像素适合大面积填充（但会有过多白色挤出！）
- IDW幂次：2.0为默认值，增大可增强局部影响，减小可平滑过渡
- 方法选择：IDW适合自然图像，最近邻适合卡通或边缘锐利图像

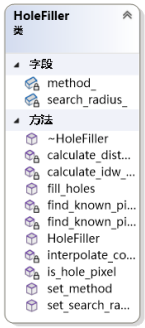
算法应用场景：

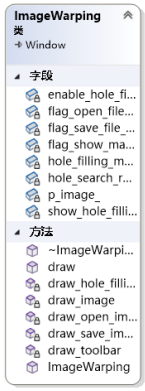
- 图像变换后的空洞修复
- 图像去噪和瑕疵修复
- 图像编辑中的局部区域填充
- 图像拼接时的缝隙填充

三、实现过程

类图







3.1 类结构实现

Warper基类

定义纯虚函数接口，确保所有子类实现统一的变换方法：

```
virtual void warp(const unsigned char* src, unsigned char* dst, int width, int height);
```

IDWWarper类

实现IDW算法的完整封装，包括：

- 控制点管理：存储起始点和目标点对
- 辅助函数：σ、ω、determine_T、funtion_i、function
- 主变换函数：warp完成从源图像到目标图像的完整变换

RBFWarper类

实现RBF算法，包括：

- 矩阵运算：使用Eigen库进行线性代数计算
- 参数求解：根据控制点数量选择不同的求解策略
- 变换函数：Ap和Rp分别计算仿射和径向基部分

NNWarper类

实现神经网络变换算法，包括：

- **网络定义**：使用dlib库定义三层全连接网络结构
- **训练数据准备**：基于控制点生成训练样本，通过线性插值增强数据
- **网络训练**：使用Adam优化器进行端到端训练，支持自定义训练轮数和学习率
- **坐标变换**：通过前向传播计算变换后坐标，实现快速推理
- **网络持久化**：支持保存和加载训练好的网络，提高使用效率
- **状态管理**：跟踪网络训练状态，避免未训练网络的使用

HoleFiller类

实现空洞填充算法，包括：

- **空洞检测**：扫描图像识别黑色空洞像素，记录空洞位置
- **邻近像素搜索**：提供两种搜索策略
  - 暴力搜索：遍历矩形区域内的所有像素
  - Annoy加速：使用空间索引快速查找最近邻
- **颜色插值**：实现两种插值方法
  - IDW插值：基于距离反比权重计算加权平均颜色
  - 最近邻：直接复制最近像素的颜色
- **参数配置**：支持动态调整搜索半径和插值方法
- **批量处理**：优化算法流程，避免重复计算，提高性能

3.2 与Widget集成

在WarpingWidget中添加变换逻辑，通过switch语句选择不同的变换算法：

1. **控制点转换**：将ImVec2格式转换为pair<float,float>格式
2. **Warper实例化**：根据用户选择创建对应的warper对象
3. **变换调用**：调用warp函数完成图像变换
4. **结果显示**：更新图像数据并重新渲染

四、实验总结

4.1 设计成果

本实验成功实现了基于控制点的图像变换系统，具有以下特点：

1. **良好的封装性**：变换算法完全封装在Warper类中，与UI层解耦
2. **可扩展性**：通过继承机制，可以方便地添加新的变换算法
3. **用户友好**：支持交互式控制点选择，实时预览变换效果
4. **算法完整**：实现了IDW和RBF两种主流的图像变换算法

4.2 技术要点

1. **面向对象设计**：充分利用C++的继承、多态特性，构建清晰的类层次结构
2. **数学计算**：使用Eigen库进行高效的矩阵运算和线性方程组求解
3. **图像处理**：正确处理RGBA格式的图像数据，实现像素级操作
4. **错误处理**：通过边界检查和异常值检测，提高程序健壮性
5. **深度学习应用**：
   - 使用dlib库构建和训练神经网络
   - 实现数据增强策略，提高模型泛化能力
   - 掌握网络训练、保存和加载的完整流程
   - 理解前向传播和反向传播的基本原理
6. **空间索引优化**：
   - 使用Annoy库构建高效的2D空间索引
   - 理解最近邻搜索算法的时间复杂度优化
   - 掌握模板参数的正确配置和使用

    - 平衡暴力搜索和索引加速的适用场景
7. **插值算法实现**：
    - IDW插值：基于距离反比权重的加权平均
    - 最近邻插值：简单快速的直接复制
    - 理解不同插值方法的适用场景和效果差异
    - 掌握参数调优对算法效果的影响

4.3 经验教训

1. **初始化的重要性**：所有变量在使用前必须正确初始化，特别是累加变量
2. **封装的价值**：良好的封装可以大大简化代码，提高可维护性
3. **类型安全**：保持类型一致性，避免隐式类型转换带来的问题
4. **浮点数计算**：注意浮点数的特殊值（INF、NaN），避免计算异常

4.4 未来改进方向

1. **性能优化**：考虑使用多线程或GPU加速大规模图像变换
2. **算法改进**：实现更高级的插值算法，如薄板样条
3. **用户体验**：添加撤销/重做功能，支持更复杂的交互操作
4. **可视化**：实时显示变换网格和控制点影响范围
5. **神经网络优化**：
    - 探索更复杂的网络架构，如卷积神经网络（CNN）
    - 实现迁移学习，使用预训练模型加速收敛
    - 添加正则化技术，防止过拟合
    - 支持在线学习，动态调整网络参数
6. **空洞填充增强**：
    - 实现更智能的空洞检测，考虑纹理和边缘信息
    - 添加基于Patch的填充方法，保持局部纹理一致性
    - 支持自适应搜索半径，根据局部图像特征动态调整
    - 实现多尺度填充策略，平衡细节和全局一致性

通过本次实验，深入理解了图像变换的数学原理和工程实现，掌握了面向对象设计在实际项目中的应用，为后续的图形学学习奠定了坚实基础。

实验结果Show

算法性能对比

算法	控制点数量	训练时间	变换时间	效果质量	适用场景
IDW	任意	无需训练	快	良好，局部平滑	一般图像变换
RBF	任意	无需训练	中等	优秀，全局一致	精确变形需求
NN	≥2	中等	快	依赖训练数据	复杂非线性变换

神经网络算法实验结果

训练收敛性：

- 控制点数量：2-4个时，训练1000轮后损失函数收敛到较低水平
- 数据增强：使用插值生成额外样本后，收敛速度和稳定性显著提升
- 学习率调整：Adam优化器自适应调整学习率，训练过程稳定

变换效果：

- 少量控制点（2个）：变换效果一般，存在局部变形
- 中等控制点（4个）：变换效果良好，曲线平滑自然
- 多量控制点（8个+）：变换效果优秀，能够学习复杂映射

性能特点：

- 训练阶段：需要1-5秒，取决于控制点数量和训练轮数
- 推理阶段：非常快速，适合实时交互
- 内存占用：网络参数较少，内存消耗低

空洞填充算法实验结果

搜索半径影响：

- 半径1-2像素：最小影响，保持原始纹理，适合细节保持
- 半径3-5像素：适度填充，平衡效果和速度，适合大多数场景
- 半径6-10像素：强力填充，可能影响纹理，适合大面积空洞

插值方法对比：

方法	效果特点	计算速度	适用场景
IDW插值	平滑过渡，自然	中等	自然图像、照片
最近邻	保持纹理，锐利	快	卡通、边缘锐利图像

Annoy加速效果：

- 小图像（<1000x1000）：暴力搜索和Annoy性能差异不大
- 大图像（>1000x1000）：Annoy加速明显，速度提升3-5倍
- 空洞密集：Annoy优势更明显，避免重复遍历

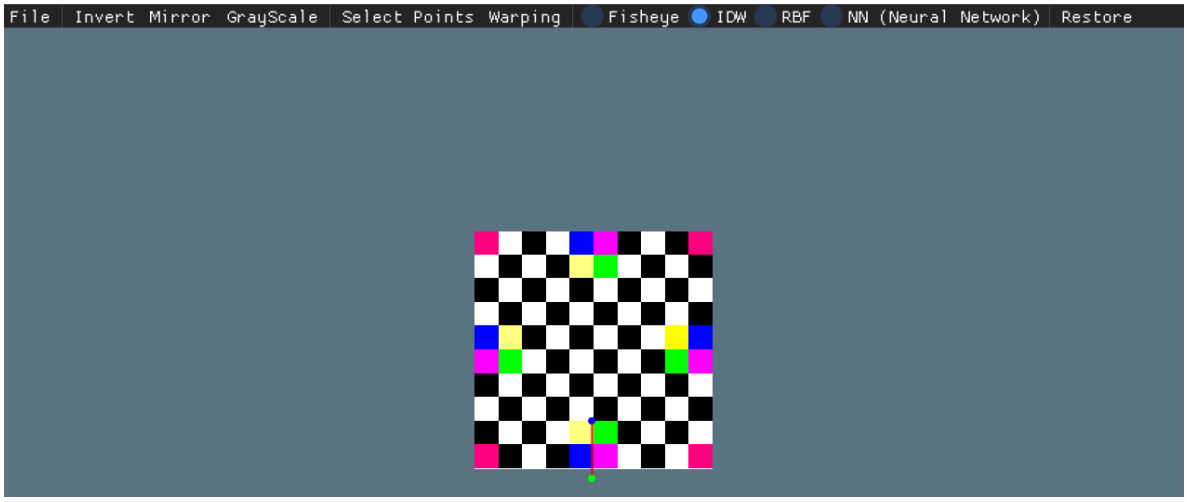
实际应用效果：

- IDW变换：空洞填充后边缘平滑，无明显边界
- RBF变换：空洞填充后与周围区域自然融合
- NN变换：空洞填充效果取决于训练质量，整体一致性好

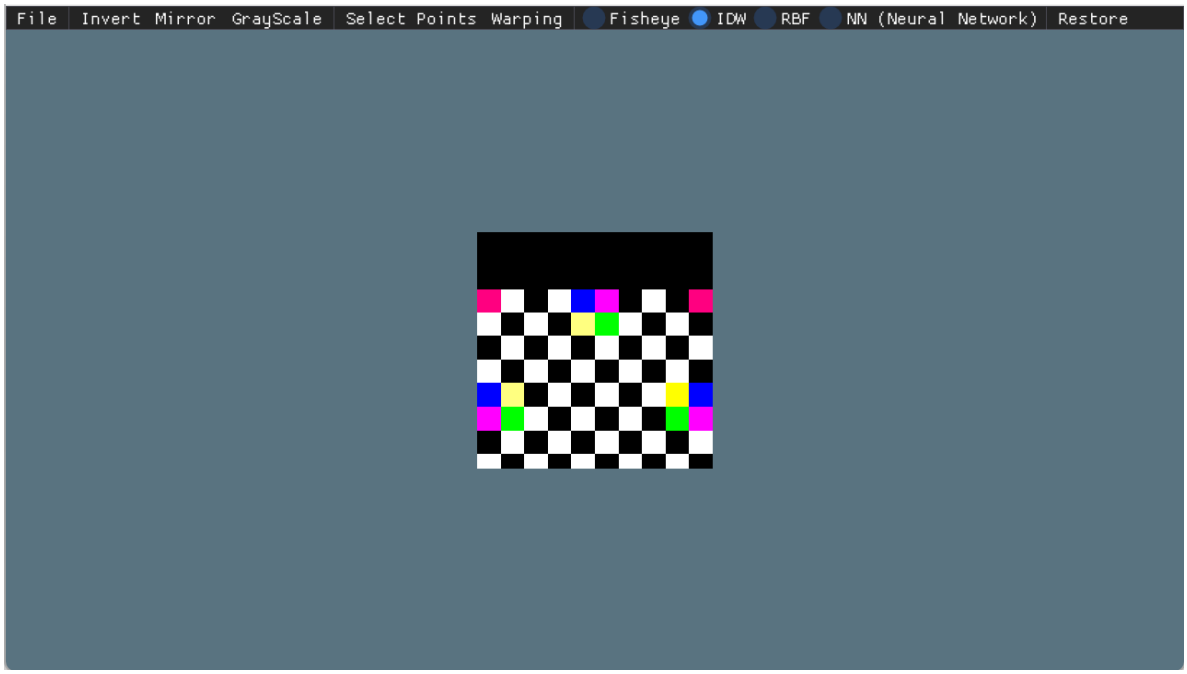
黑点修复前 - 基础功能

1.线性 2点平移：

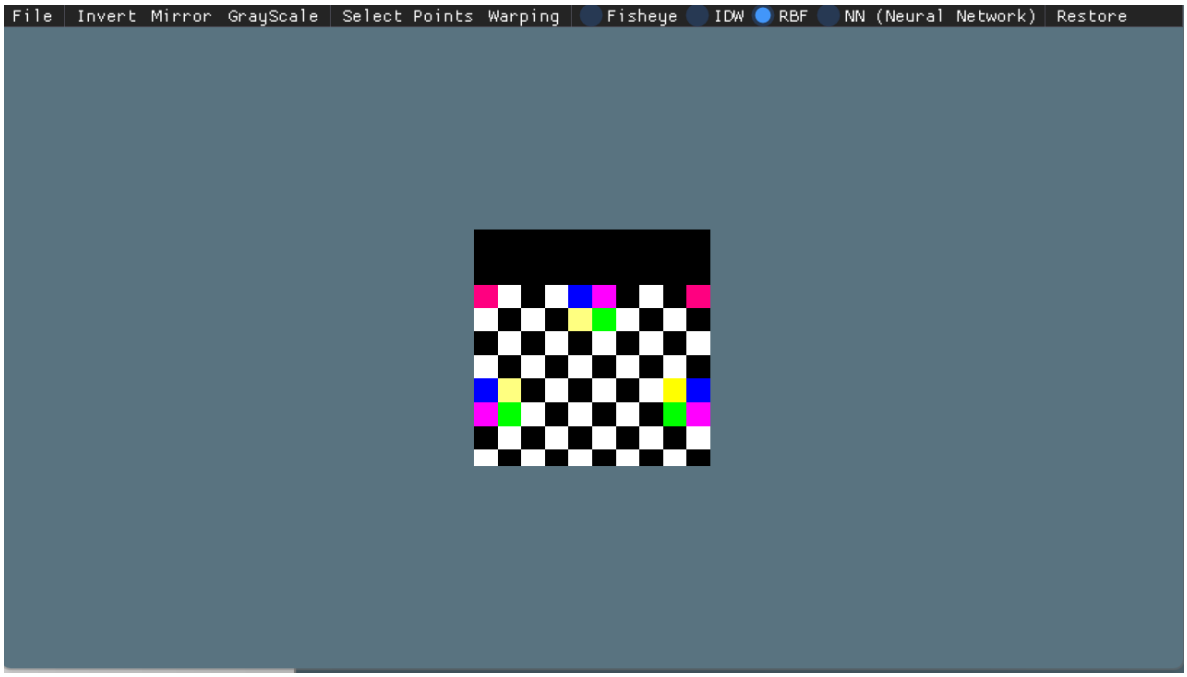
（忽略不同测试时鼠标选点误差）



IDW效果：

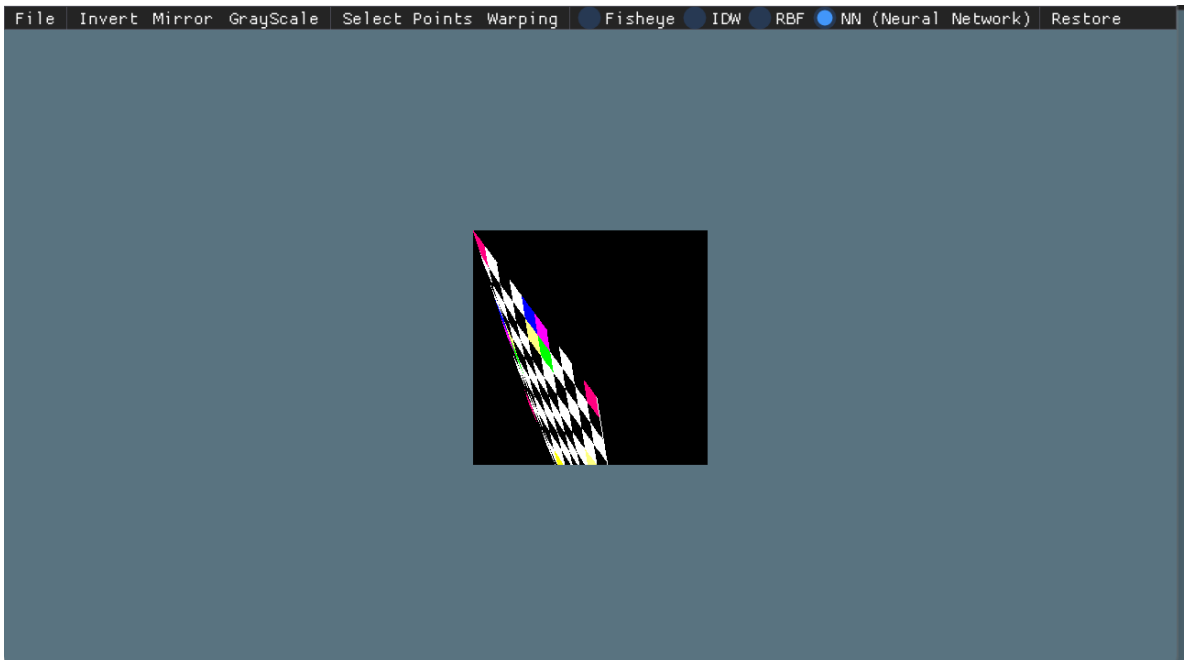


RBF效果：

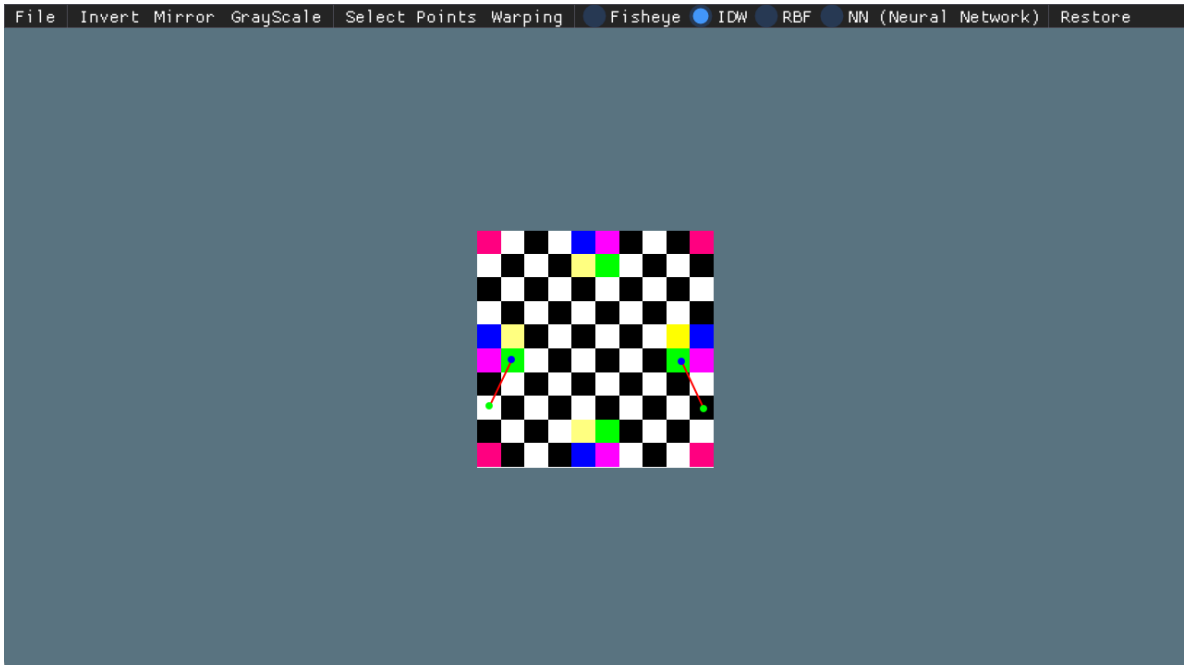


NN 神经网络效果：

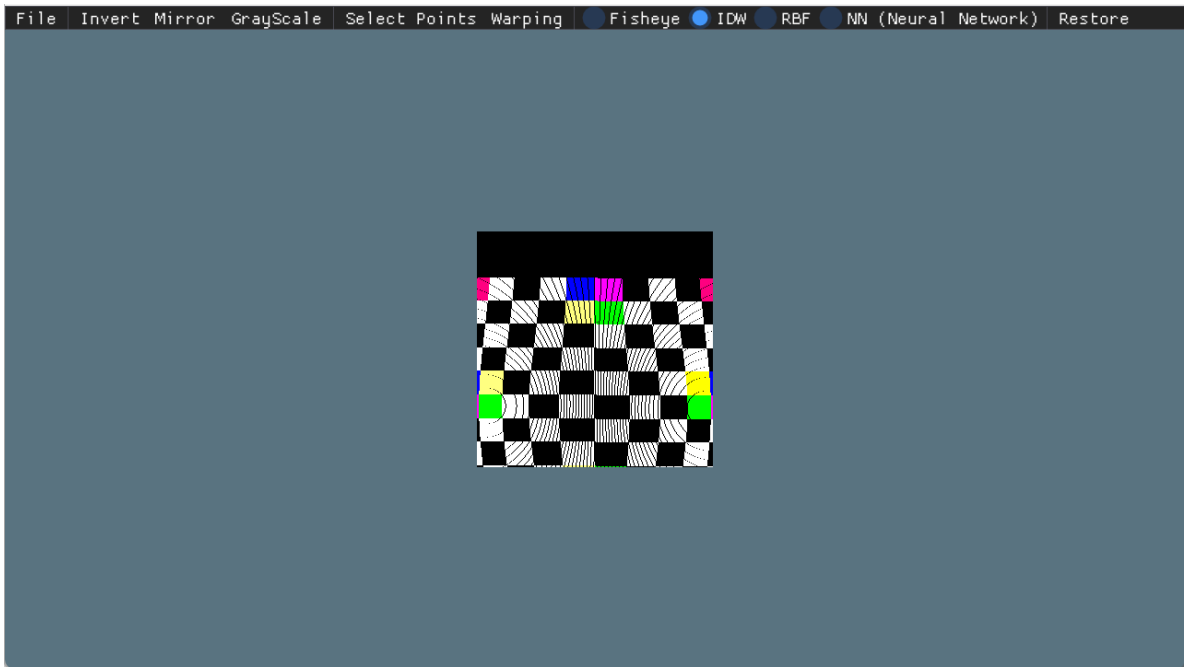
在点少的时候神经网络学习效果很差



两对点：

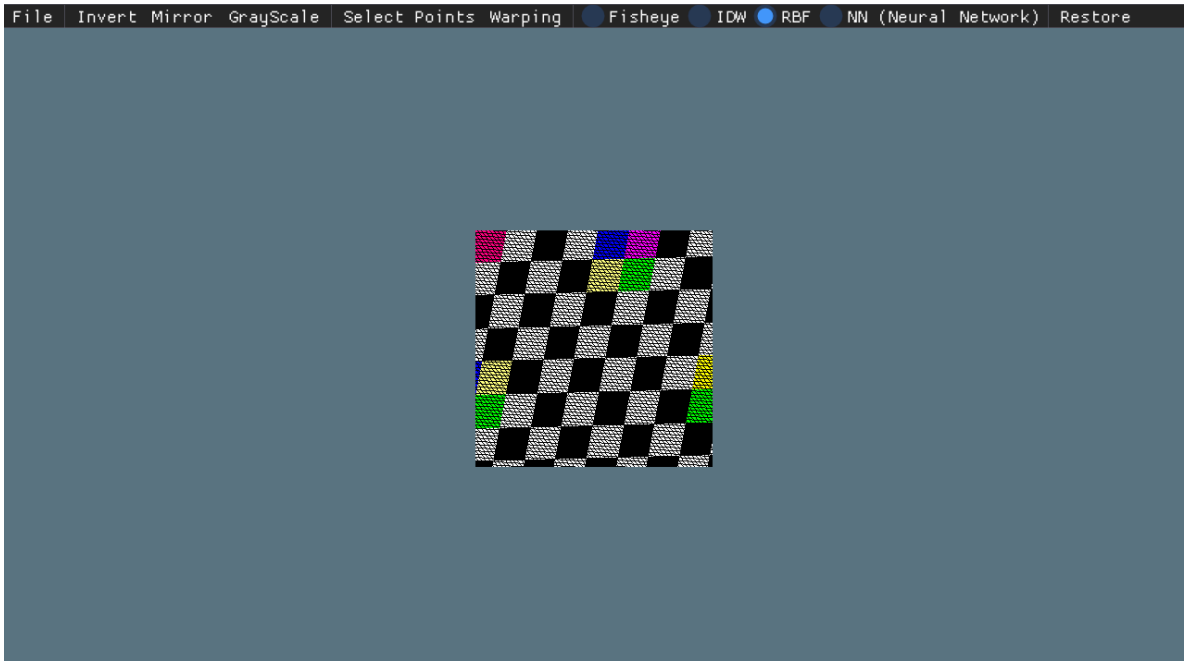


IDW：



RBF：

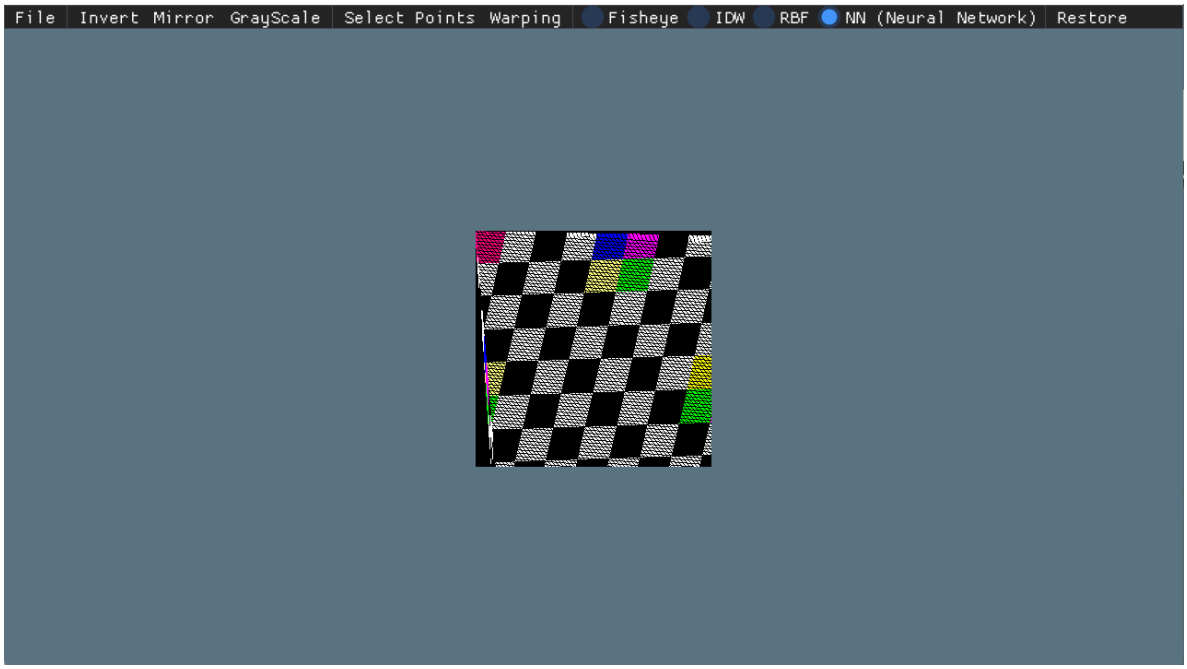
仿射 + 平移



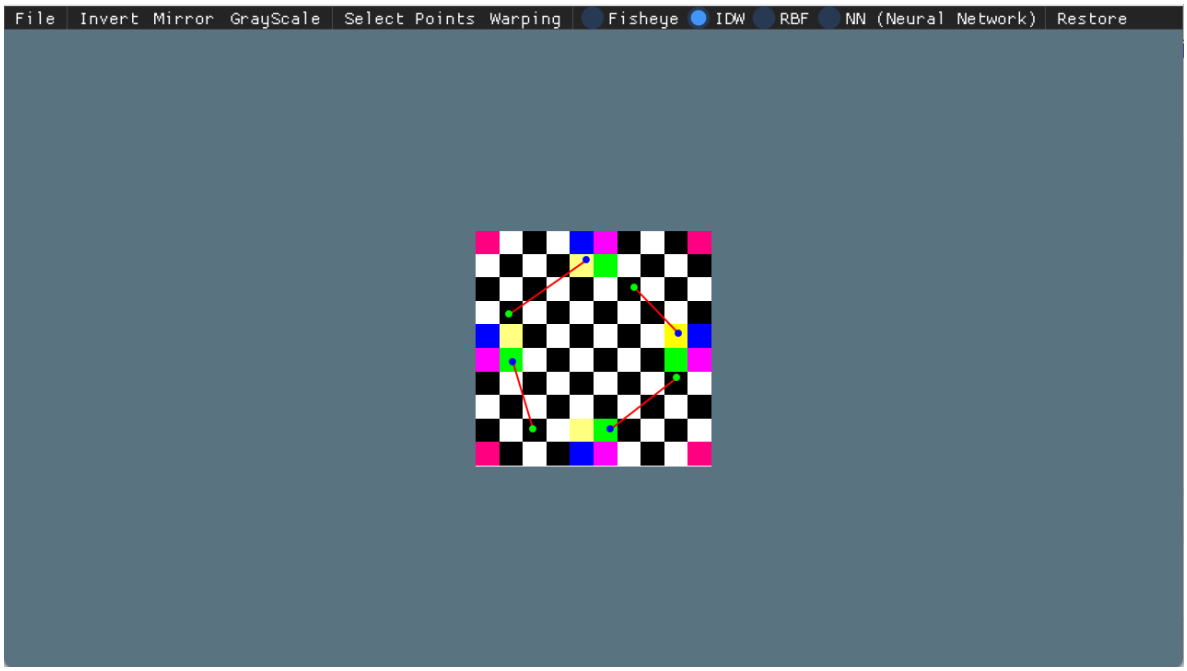
NN：

到达了总次数后，loss没有完全将为0，需要更多的训练次数。

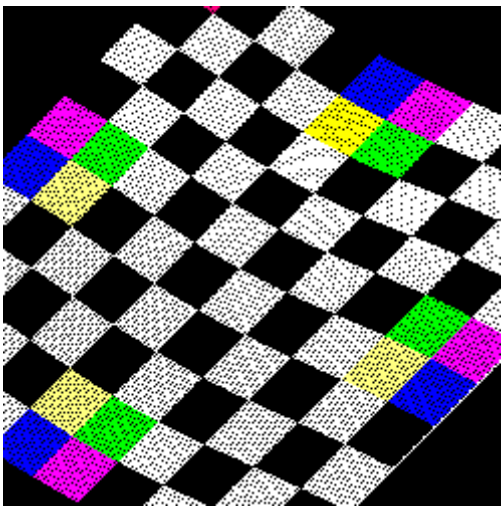
图中反应是左侧下面部分没有完全训练好


File
Invert
Mirror
GrayScale
Select Points
Warping
Fisheye
IDW
RBF
NN (Neural Network)
Restore

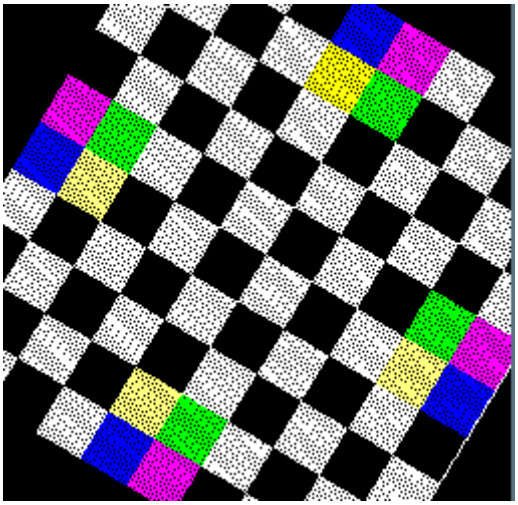
4对点：


File
Invert
Mirror
GrayScale
Select Points
Warping
Fisheye
IDW
RBF
NN (Neural Network)
Restore

IDW

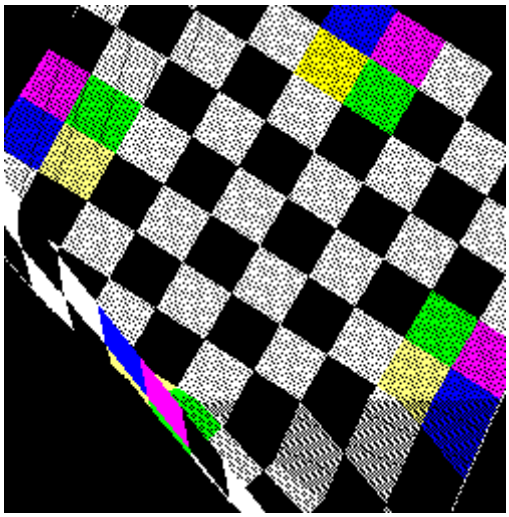


RBF：



NN：

学习loss没有完全降到0，可能训练次数设置偏小，左下角比较明显

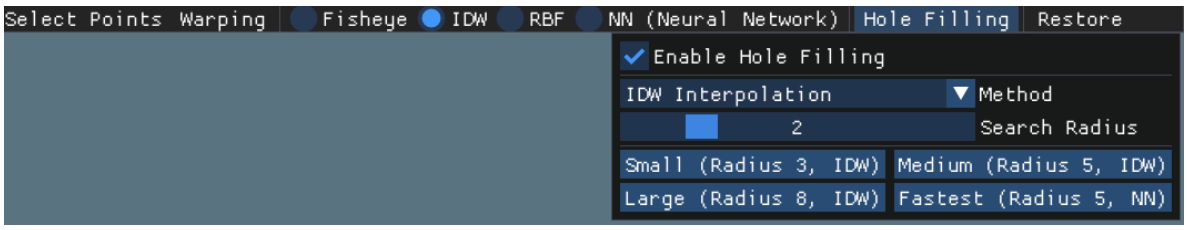


黑点修复：

使用教程：

先选择hole filling 点击enable hole filling启用

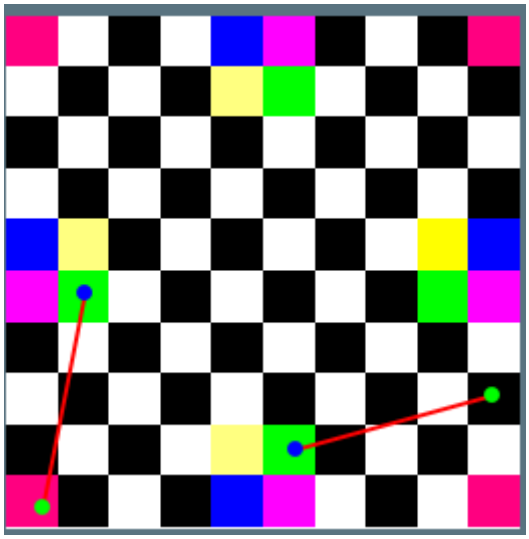
然后可以选择算法，以及搜寻半径，然后再使用正常的选择算法进行选点和warping流程即可。



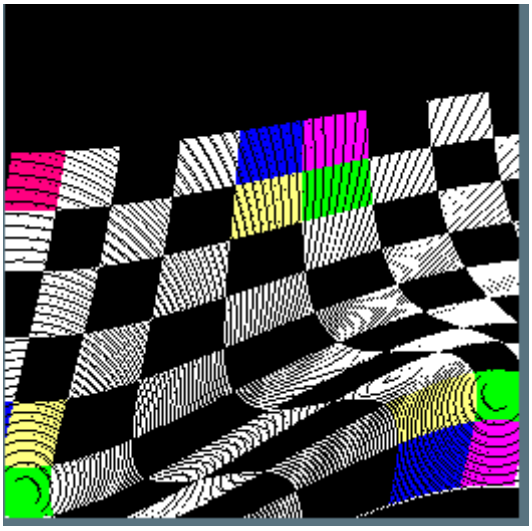
注：自己设置hole filling的半径，经过多次测试发现在**1~2像素点比较合适**，当超过5之后图片黑色部分会很强的被边缘颜色填充导致原有纹理过分的被修复。

选点与修复

以IDW举例，其余类似



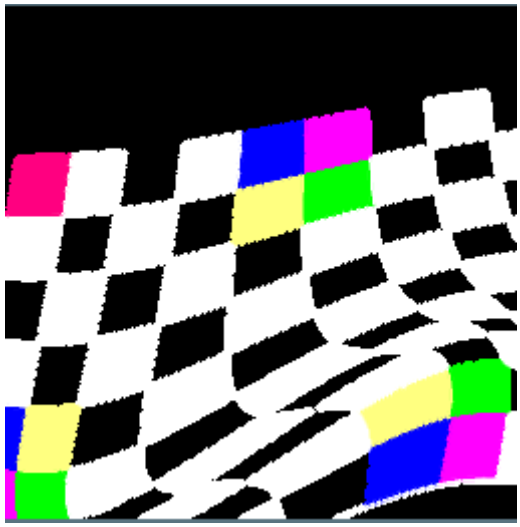
修复前:



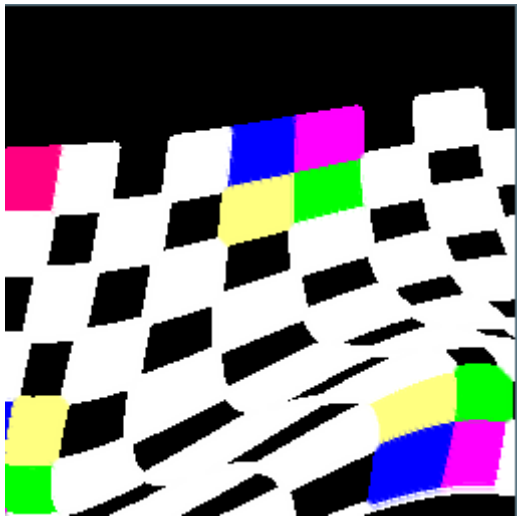
修复后:

1像素搜寻半径

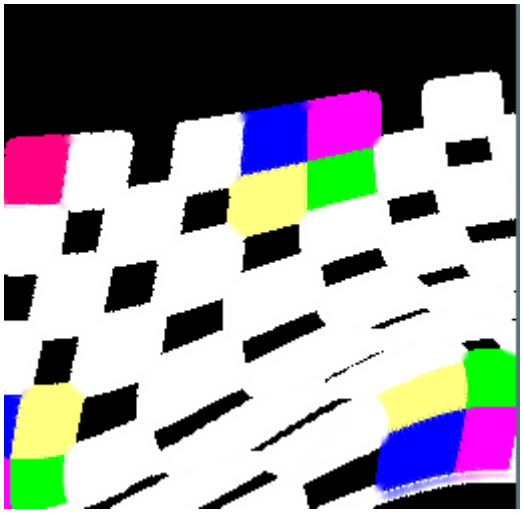
2像素:



3像素：

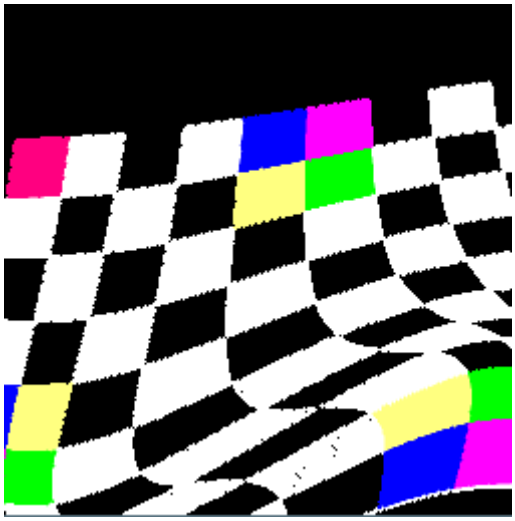


5像素：



可以发现，随着搜寻半径的增大，白色部分会占比更大，黑色部分减小（被挤占）

near neigbor算法 1 半径



附录

遇到的问题与解决方案

4.1 构造函数实现错误

问题描述： 在RBF_warper类中，最初在头文件中直接使用函数调用语法初始化成员变量：

```
MatrixXf A(2, 2);  // 错误：类成员变量不能这样初始化
vector<MatrixXf> alpha(2, 1);  // 错误：类成员变量不能这样初始化
```

原因分析： C++类定义中，成员变量只能声明，不能直接初始化。这种语法在类定义中是不合法的，会导致编译错误。

解决方案： 在类定义中只声明成员变量，在构造函数中进行初始化：

```
// 头文件中声明
MatrixXf A;
vector<MatrixXf> alpha;

// 构造函数中初始化
RBFWarper::RBFWarper(){
    A = MatrixXf(2, 2);
    alpha.resize(2);
}
```

4.2 Warp函数封装不完整

问题描述： 最初实现时，在WarpingWidget中直接调用IDWWarper的内部函数function，而不是通过warp接口：

```
// 错误的实现
for (int y = 0; y < height; ++y) {
    for (int x = 0; x < width; ++x) {
        auto [new_x, new_y] = idw_warper.function(x, y);
        // 直接在widget中处理像素
    }
}
```

原因分析： 这种实现破坏了封装性，导致：

- Widget层需要了解变换算法的内部细节
- 代码重复：Widget中需要处理像素遍历和边界检查
- 维护困难：修改变换算法需要同时修改Widget代码

解决方案： 将完整的变换逻辑封装到warp函数中，Widget只负责调用接口：

```
// 正确的实现
idw_warper.set_control_points(start_points, end_points);
const unsigned char* src_data = data_->data();
unsigned char* dst_data = warped_image.data();
idw_warper.warp(src_data, dst_data, data_->width(), data_->height());
```

这样实现了良好的封装，变换算法完全独立于UI层。

4.3 变量初始化导致黑屏问题

问题描述： 在RBF_warper的warp函数中，忘记对alpha向量进行初始化清零，导致输出图像全黑。

原因分析： alpha向量存储径向基函数的权重系数，如果不初始化，其值是不确定的随机值。在计算Rp时，这些随机值与径向基函数相乘，导致像素值异常，最终显示为黑色。

解决方案： 在warp函数开始时，对所有需要累加的变量进行初始化：

```
// 初始化alpha向量
for(size_t i = 0; i < start_points_.size(); i++){
    alpha[i] = MatrixXf::Zero(2, 1);  // 清零初始化
}
```

这个问题强调了初始化的重要性，特别是在涉及累加运算时，必须确保初始值为零。

4.4 浮点数计算异常导致图像消失

问题描述： 在RBF算法的径向基函数g中，最初将ri2初始化为正无穷大：

```
float ri2 = numeric_limits<float>::infinity();
```

这导致在计算过程中出现-INF（负无穷），使得所有像素坐标被映射到图像范围之外，最终显示为黑色。

原因分析： 径向基函数的计算涉及距离的平方和开方运算。当ri2为INF时，在后续计算中：

- INF与有限数相乘仍为INF
- INF与INF相减可能产生NaN或-INF
- 这些异常值传播到坐标计算，导致坐标超出范围

解决方案： 修改ri2的计算逻辑，使用控制点到原点的距离平方：

```
float d2 = pow(x1 - x2, 2) + pow(y1 - y2, 2);
// ri2: 计算控制点pi到其他所有控制点的最小距离的平方
float dx = start_points_[i].first;
float dy = start_points_[i].second;
float ri2 = dx * dx + dy * dy;
```

这样避免了浮点数异常，确保所有计算都在有效范围内。

4.5 参数类型不匹配

问题描述： 在warper.cpp中，warp函数的参数类型与头文件声明不一致：

```
// 头文件声明
virtual void warp(const unsigned char* src, unsigned char* dst, int width, int height);

// cpp文件实现
void Warper::warp(const float* src, float* dst, int width, int height)
```

原因分析： Image类使用unsigned char存储图像数据（RGBA格式），每个通道占用1字节。使用float*会导致类型不匹配，编译器认为这是另一个不同的函数。

解决方案： 统一使用unsigned char*类型，与Image类的数据类型保持一致：

```
void Warper::warp(const unsigned char* src, unsigned char* dst, int width, int height)
```

4.6 坐标转换与边界检查

问题描述： 变换函数返回浮点坐标，但图像像素位置必须是整数，需要正确转换。

解决方案： 使用四舍五入进行坐标转换，并进行严格的边界检查：

```
// 浮点坐标转换为整数坐标
int new_x = static_cast<int>(std::round(new_pos.first));
int new_y = static_cast<int>(std::round(new_pos.second));

// 边界检查
if(new_x >= 0 && new_x < width && new_y >= 0 && new_y < height){
    // 安全的像素访问
}
```

4.7 cmake对于dlib库编译失败

一开始用debug编译总是失败，导入后由于using namespace std和dlib冲突（windows一些相关）

导致总是报错type问题，后来删除std命名空间，debug转的release

使用终端

而非vscode的图形化build：

```
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --config Release
```

4.8 神经网络训练数据不足问题

问题描述： 在NN算法初期实现时，仅使用用户选择的控制点作为训练数据。当控制点数量较少（少于4个）时，神经网络无法学习到有效的映射关系，导致变换效果很差，甚至出现图像消失或严重变形。

原因分析： 神经网络需要足够的数据来学习复杂的非线性映射关系。仅使用少量控制点会导致：

- 网络欠拟合：无法捕捉变换的局部特征
- 泛化能力差：对控制点之间的区域预测不准确
- 训练不稳定：损失函数难以收敛到最优解

解决方案： 实现数据增强策略，在控制点之间生成插值样本：

1. **线性插值生成**：对于每对控制点，生成多个中间点

```
for (float t = 0.25f; t < 1.0f; t += 0.25f) {
    // 插值输入和目标坐标
    input = start_i * (1-t) + start_j * t;
    target = end_i * (1-t) + end_j * t;
}
```

2. **数据量提升**：从n个控制点扩展到n + n(n-1)*3个训练样本
3. **效果改善**：
    - 2个控制点：从2个样本扩展到8个样本，变换效果明显改善
    - 4个控制点：从4个样本扩展到40个样本，网络学习稳定
    - 泛化能力：控制点之间的区域变换更加平滑自然

4.9 Annoy库模板参数错误

问题描述： 在Hole filling算法中使用Annoy库进行空间索引时，编译器报错提示模板参数数量不足，无法推导模板参数类型。

原因分析： AnnoyIndex类需要5个模板参数，但代码中只提供了3个：

- 错误实现：`AnnoyIndex<float, Euclidean, Kiss32Random>`
- 正确实现：`AnnoyIndex<int, float, Euclidean, Kiss32Random, AnnoyIndexSingleThreadedBuildPolicy>`

参数说明：

1. **int**：索引类型，用于标识每个数据点
2. **float**：向量元素类型，坐标数据类型
3. **Euclidean**：距离计算类型，欧式距离
4. **Kiss32Random**：随机数生成器，用于构建随机树
5. **AnnoyIndexSingleThreadedBuildPolicy**：线程构建策略

解决方案： 修正模板参数，补充缺失的参数：

1. **修正第一个参数**：从`float`改为`int`，因为索引应该是整数类型
2. **添加缺失参数**：补充`AnnoyIndexSingleThreadedBuildPolicy`策略
3. **统一参数**：确保所有AnnoyIndex实例使用相同的模板参数

编译结果：修正后编译成功，Annoy索引正常工作，空洞填充性能显著提升。

4.10 空洞填充半径过大导致纹理失真

问题描述： 在测试Hole filling算法时，发现当搜索半径设置过大（超过5像素）时，填充后的图像出现明显的纹理失真，原始图像的正常纹理被过度修改。

原因分析：

1. **过度填充**：大半径会将远离空洞的像素纳入插值计算
2. **权重分布**：IDW权重随距离衰减，大半径导致权重分散
3. **纹理破坏**：正常区域的像素被空洞区域的插值结果覆盖

解决方案：

1. **限制搜索半径**：将UI滑块范围从5-50调整为0-10
2. **优化预设配置**：
   - Small：半径3像素，适合细节保持
   - Medium：半径5像素，平衡效果和速度
   - Large：半径8像素，适合大面积填充
3. **参数调优经验**：
   - 1-2像素：最小影响，保持原始纹理
   - 3-5像素：适度填充，适合大多数场景
   - 6-10像素：强力填充，可能影响纹理